

SANTO HOTEL

PHASE 2

Database & Domain Model

INTERN TRAINING & IMPLEMENTATION GUIDE

Project	Santo Hotel — Single-Tenant Hotel Booking Web Application
Phase	2 — Database & Domain Model
Audience	Software Engineering Intern / Junior Developer
Purpose	Translate business requirements into a reliable database/domain model
Suggested Duration	3–5 working days
Prerequisite	Phase 1 approved by mentor
Mobile App	Not part of this phase

Training principle: design the data model before writing business logic.

1. Phase Overview

Phase 2 converts the business understanding from Phase 1 into a structured domain model and database design. The intern is not simply creating tables; they are deciding how Santo Hotel's real-world objects, relationships, rules and history will be represented reliably.

A weak database design creates problems later in availability, bookings, payments and reporting. A good design makes those features easier to build, test and maintain.

PHASE 2 GOAL — By the end of this phase, the intern should explain every core entity, its relationships, important fields, constraints and why the database is structured that way.

2. Learning Objectives

- Identify entities from business requirements.
- Distinguish entities from attributes and actions.
- Define primary keys and foreign keys.
- Understand one-to-one, one-to-many and many-to-many relationships.
- Create an Entity Relationship Diagram (ERD).
- Normalize data to avoid unnecessary duplication.
- Choose appropriate field types.
- Design constraints that protect data integrity.
- Create version-controlled database migrations.
- Create safe development seed data.
- Add useful indexes.
- Prepare the schema for availability and booking queries.

3. Start With the Domain, Not the Tables

Before creating migrations, describe the hotel domain in plain language.

```
Santo Hotel
has room types
has physical rooms
has amenities
has rates
has guests
receives bookings
receives payments
offers promotions
sends notifications
records important actions
```

These statements become candidates for domain entities. The database design should represent the business rather than being driven only by what is convenient to code.

TRAINING RULE — Do not create a table simply because a word appears in the PRD. First decide whether it is an entity, attribute, action, derived value or configuration.

4. Entity vs Attribute vs Action

Entity

A meaningful business object that usually needs its own identity and stored record.

Guest
Room
Room Type
Booking
Payment

Attribute

A property describing an entity.

Guest
■■■ first_name
■■■ last_name
■■■ email
■■■ phone

Action

Something the application does to or with entities.

Create Booking
Cancel Booking
Check In Guest
Process Payment

5. Core Domain Entities

Entity	Purpose
Hotel	Santo Hotel information and configuration
User	Authenticated staff/admin account
Guest	Customer associated with reservations
Room Type	Accommodation category
Room	Physical hotel inventory
Amenity	Reusable room feature
Room Image	Room photography/media reference
Rate	Price applicable to a room type and period
Booking	Reservation header and commercial record
Booking Room	Room allocation within a booking
Payment	Financial transaction
Promotion	Discount/promotion rule
Notification	Message/event sent to a user or guest
Audit Log	History of important administrative actions

6. Most Important Relationships

Hotel → Room Types

One Hotel
↓
Many Room Types

Hotel 1 ■■■■■■< Room Types

Room Type → Physical Rooms

Deluxe King
■■■ Room 101
■■■ Room 102
■■■ Room 103

One Room Type → Many Rooms

Guest → Bookings

Guest 1
■■■ Booking A
■■■ Booking B
■■■ Booking C

One Guest → Many Bookings

Booking → Booking Rooms → Physical Rooms

Booking A
■■■ Booking Room → Room 101
■■■ Booking Room → Room 102

This allows a single reservation to contain multiple physical rooms.

Booking → Payments

A booking may have multiple payment records depending on the payment/refund design. Do not assume that one booking always equals one transaction.

7. Relationship Types

One-to-One

User 1 ■■■■ 1 Profile

Use only when the business relationship genuinely requires it.

One-to-Many

Room Type 1 ■■■■< Rooms

One room type can have many physical rooms.

Many-to-Many

Room Types >■■■■< Amenities

```
room_type_amenities
-----
room_type_id
amenity_id
```

A junction table represents a many-to-many relationship.

TRAINING QUESTION — Why not store 'Wi-Fi, TV, AC' in one text field? Because searching, validating, updating and relating individual amenities becomes difficult.

8. Primary Keys

Every major entity needs a stable identifier.

```
guests
id

rooms
id

bookings
id

payments
id
```

The database identifier can be separate from the customer-facing booking reference.

Internal database ID:
8f2... / 12345

Customer booking reference:
SH-2026-000123

DESIGN PRINCIPLE — A customer-facing booking reference does not need to be the database primary key.

9. Foreign Keys

Foreign keys connect related records and protect referential integrity.

```
bookings
id
guest_id → guests.id

rooms
id
room_type_id → room_types.id

booking_rooms
booking_id → bookings.id
room_id → rooms.id
```

The database should not allow a foreign key to reference a record that does not exist, unless the relationship is intentionally nullable.

10. Booking Data Model

```
bookings
-----
id
booking_reference
guest_id
check_in
check_out
adults
children
subtotal
taxes
fees
discount
total
booking_status
payment_status
created_at
updated_at

booking_rooms
-----
id
booking_id
```

```
room_id
nightly_rate
nights
room_total
```

The exact financial fields should follow the approved hotel policy and payment design.

Booking-level information belongs on the booking. Room-specific allocation and historical room pricing belong on the booking-room record when appropriate.

11. Why Historical Booking Prices Matter

If a room costs RM450 when a guest books it, and the hotel later changes the current rate to RM500, the old booking must remain RM450.

```
Current configuration:
Room rate = RM500
```

```
Historical transaction:
Booking rate = RM450
```

This is why transactional records often store the price actually used at the time of the transaction.

TRAINING PRINCIPLE — Current configuration and historical transaction data are different things.

12. Room Type vs Physical Room

```
room_types
-----
id
name
description
max_adults
max_children
base_price
status

rooms
-----
id
room_type_id
room_number
floor
status
```

Room type contains characteristics shared by the category. Physical room contains inventory-specific information.

TRAINING QUESTION — If Santo Hotel has 10 Deluxe King rooms, should there be 10 room types? No. Normally there is one Deluxe King room type and 10 physical rooms.

13. Amenities

```
amenities
-----
id
name
description

room_type_amenities
-----
room_type_id
amenity_id
```

For MVP, amenities can normally be associated with room types. If Santo Hotel later needs room-specific amenities, the model can be extended.

14. Rates & Pricing

Pricing may eventually include base, seasonal, weekend and special-date rates. MVP should implement only what the approved requirements need.

```
rates
-----
id
room_type_id
name
amount
currency
start_date
end_date
status
```

- The amount should use a suitable decimal/numeric type.
- Currency should be explicit if multiple currencies could ever be supported.
- The booking should preserve the rate actually applied.
- Do not build advanced dynamic revenue management in this phase.

SCOPE CONTROL — The intern should build the pricing model required by the MVP, not an entire hotel revenue-management platform.

15. Promotions

```
promotions
-----
id
code
type
value
start_at
end_at
max_uses
status
```

A booking should preserve the discount actually applied. An expired promotion must not rewrite historical bookings.

16. Payment Data

```
payments
-----
id
booking_id
provider
provider_reference
amount
currency
status
paid_at
refunded_at
created_at
```

- Never store raw card numbers.
- Store the provider reference.
- Store payment status.

- Record transaction timestamps.
- Support provider webhook reconciliation.
- Keep payment records auditable.

Exact fields depend on the selected payment provider.

17. Normalization

Normalization reduces unnecessary duplication and prevents conflicting copies of the same business information.

Bad Structure

```
rooms
room_number | room_type | description | price
101 | Deluxe | ... | 450
102 | Deluxe | ... | 450
103 | Deluxe | ... | 450
```

Better Structure

```
room_types
id | name | description | price

rooms
id | room_number | room_type_id
```

Shared room-type information is stored once and physical rooms reference it.

TRAINING RULE — Avoid accidental duplication. If a value belongs to a shared business object, consider whether it should live in that object's table.

18. Intentional Historical Snapshots

Not all repeated values are bad. Transaction records often need a snapshot of what was true when the transaction occurred.

```
room_types.base_price = current configuration

booking_rooms.nightly_rate = historical booking price
```

The current rate can change while the historical booking remains correct.

The intern must distinguish accidental duplication from intentional historical data.

19. Constraints & Data Integrity

- Use NOT NULL for genuinely required fields.
- Use UNIQUE constraints for values that must be unique.
- Use foreign keys for relationships.
- Use suitable decimal types for money.
- Use controlled status values.
- Make room numbers unique within the hotel where appropriate.
- Protect booking references from duplicates.
- Define deletion behavior for related records.

DATABASE PRINCIPLE — Critical integrity should be protected by the database as well as by application code.

20. Data Types

Data	Typical Choice
Identifier	UUID or BIGINT
Name	VARCHAR
Description	TEXT
Money	DECIMAL / NUMERIC
Date	DATE
Timestamp	TIMESTAMP / TIMESTAMPTZ
Boolean	BOOLEAN
Count	INTEGER
Status	ENUM or controlled value
URL	VARCHAR / TEXT

The exact choice between UUID and integer IDs should follow the project's architecture and conventions. Consistency is more important than fashion.

21. Indexing

Indexes help the database locate frequently searched records efficiently.

Likely Useful Indexes

- bookings.booking_reference
- bookings.guest_id
- bookings.check_in
- bookings.check_out
- bookings.booking_status
- rooms.room_type_id
- rooms.status
- booking_rooms.booking_id
- booking_rooms.room_id
- payments.booking_id
- payments.provider_reference

Do not index every field. Indexes consume storage and can add write overhead.

TRAINING EXERCISE — For every index, identify the query it is intended to improve.

22. Designing for Availability

The availability engine will later need to answer questions such as which physical rooms are eligible for a requested date range.

```
Find rooms
WHERE room is active
AND room is not maintenance/out-of-service
AND room is not allocated to an overlapping booking
```

Phase 2 does not implement the availability engine. It makes sure the schema can support it.

Overlap Thinking

```
Requested:
[ 15 ----- 18 ]

Existing:
[ 17 ----- 20 ]

Overlap = YES

Requested:
[ 18 ----- 20 ]

Existing:
[ 15 ----- 18 ]

Overlap = NO
if check-out/check-in on the same date is allowed.
```

The exact rule must follow Santo Hotel's approved check-in/check-out policy.

23. Audit Logs

```
audit_logs
-----
id
user_id
action
entity_type
entity_id
old_values
new_values
created_at
```

Examples include an administrator changing a room rate or a staff member cancelling a booking.

Audit data is for accountability/history. It should not replace the normal application state.

24. Migrations

Database changes should be represented by version-controlled migrations so the schema can be recreated consistently.

```
001_create_hotels
002_create_users
003_create_guests
004_create_room_types
005_create_rooms
006_create_amenities
007_create_bookings
008_create_booking_rooms
009_create_payments
...
```

- Respect foreign-key dependency order.
- Keep migrations in source control.
- A clean database should be buildable from migrations.
- Do not make undocumented manual production schema changes.
- Review destructive migrations carefully.

25. Seed Data

Seed data gives developers a realistic local environment.

```
Hotel:
Santo Hotel

Room Types:
Deluxe King
Executive Suite
Family Room

Rooms:
101
102
103
201
202

Amenities:
Wi-Fi
Air Conditioning
TV
Mini Fridge
Private Bathroom
```

- Use fake/development data only.
- Create enough rooms to test availability.
- Create multiple room types.
- Create realistic rates.
- Avoid real customer information.

26. Recommended Implementation Order

```
1. Review Phase 1 deliverables
↓
2. Build domain/entity list
↓
3. Draw ERD
↓
4. Review relationships
↓
5. Define fields and constraints
↓
6. Normalize
↓
7. Plan indexes
↓
8. Create migrations
↓
9. Create seed data
↓
10. Run integrity tests
↓
11. Mentor approval
```

27. Practical Exercises

Exercise A — Entity List

List all MVP entities and explain why each exists.

Exercise B — ERD

Draw Hotel, Room Type, Room, Guest, Booking, Booking Room, Payment, Amenity, Rate and Promotion with correct cardinality.

Exercise C — Keys

For each entity, identify the primary key and foreign keys.

Exercise D — Normalize

```
room_number | room_type | room_description | amenity_1 | amenity_2 | price
101 | Deluxe | ... | Wi-Fi | TV | 450
102 | Deluxe | ... | Wi-Fi | TV | 450
```

Redesign this into a normalized structure.

Exercise E — Historical Snapshot

Identify which booking values must remain unchanged if hotel configuration changes later.

Exercise F — Index Justification

Choose at least five indexes and explain the query each supports.

Exercise G — Migration Order

Write the order in which tables should be created so foreign-key dependencies work correctly.

28. Phase 2 Deliverables

Deliverable	Expected Result
1. Domain Entity List	Approved list of core business entities.
2. ERD	Clear relationships and cardinality.
3. Data Dictionary	Fields, meanings, types and required/optional status.
4. Constraints	PKs, FKs, unique and integrity rules.
5. Index Plan	Indexes with a reason for each.
6. Migration Plan	Ordered schema migration sequence.
7. Migrations	Working migrations in source control.
8. Seed Data	Safe, realistic development data.
9. Integrity Tests	Evidence that relationships and constraints work.
10. Open Questions	Remaining business decisions needing confirmation.

29. Mentor Review Questions

1. Why are Room Type and Room separate entities?
2. Why are Booking and Booking Room separate?
3. What is the relationship between Guest and Booking?
4. What is the relationship between Room Type and Room?
5. Why should amenities not be stored as comma-separated text?
6. Why should a booking preserve the rate used at booking time?
7. What is a primary key?

8. What is a foreign key?
9. What is a unique constraint?
10. Why do we need indexes?
11. What is normalization?
12. When can intentional duplication be correct?
13. What happens if a referenced guest is deleted?
14. How do payments relate to bookings?
15. Which tables are essential for availability queries?
16. Which business rules should the database help enforce?

30. Phase Completion Gate

Phase 2 is complete only when the mentor is satisfied that the intern understands the model, not merely that the migrations run.

- ERD has been reviewed and approved.
- Core entities have clear purpose.
- Primary and foreign keys are correct.
- Room Type vs Room is correctly modeled.
- Booking vs Booking Room is correctly modeled.
- Historical booking prices are handled correctly.
- Constraints protect important data.
- Indexes have clear justification.
- Migrations run successfully from a clean database.
- Seed data creates a usable development environment.
- Integrity tests pass.
- No unresolved critical business assumptions remain.

PHASE 2 COMPLETE — Once approved, proceed to Phase 3: Authentication & Authorization.

31. Final Mental Model

```
REAL HOTEL
↓
Business Objects
↓
Entities
↓
Relationships
↓
Database Schema
↓
Constraints
↓
Indexes
↓
Migrations
↓
Seed Data
↓
Backend Business Logic
```

The database is not just a place to store information. It represents the structure of the hotel business and protects the integrity of the application's most important data.

FINAL PRINCIPLE — A good database design makes correct behavior easier and incorrect behavior harder.